Day-5 100 Days Challenge

Reverse IP Lookup: Comprehensive Guide

1. What is Reverse IP Lookup?

Definition: Reverse IP lookup identifies all domains hosted on the same IP address. If multiple websites share one server, compromising one can potentially give access to others.

Why It Matters:

- Shared hosting = shared vulnerabilities
- One vulnerable site can be a gateway to others
- Identifies client portfolios of hosting providers
- Finds related/partner sites

2. Methods & Tools

A. Using Command Line Tools

Method 1: `host` and `dig` (Built-in Linux)

```
# Step 1: Get IP of target
host target.com
# or
dig target.com +short

# Step 2: Find other domains on same IP
dig -x 192.168.1.100
# or
host 192.168.1.100
```

Example:

```
# Get IP of abcd.com
dig abcd.com +short
# Suppose it returns: 10.21.93.107

# Reverse lookup
dig -x 10.21.93.107 +short
# or
host 10.21.93.107
```

Method 2: Using `nslookup`

```
# Get IP
nslookup target.com

# Reverse lookup
nslookup -type=ptr 10.21.93.107
```

B. Using Online Tools

Method 3: ViewDNS.info

```
# Command line version
curl "https://api.viewdns.info/reverseip/?host=10.21.93.107&apikey=YOURKEY&output=json"
```

Web Interface: https://viewdns.info/reverseip/

Method 4: SecurityTrails API

```
# Requires API key (free tier available)
curl "https://api.securitytrails.com/v1/domains/list?ipv4=10.21.93.107" \
  -H "APIKEY: your_api_key_here"
```

Method 5: HackerTarget API

bash

```
curl "https://api.hackertarget.com/reverseiplookup/?q=10.21.93.107"
```

C. Using Advanced Tools

Method 6: Subfinder with Reverse IP

```bash
# Get IP first
IP=$(dig target.com +short)

# Use subfinder's reverse IP feature
subfinder -d $IP -o reverse_results.txt
```

Method 7: Masscan + httpx

```bash
# Step 1: Find all hosts on nearby IP range
masscan -p80,443 10.21.93.0/24 --rate=1000 -oG masscan_output.txt

# Step 2: Extract IPs
cat masscan_output.txt | grep "Host:" | cut -d" " -f2 | sort -u > ips.txt

# Step 3: For each IP, try reverse DNS
for ip in $(cat ips.txt); do
    host $ip | grep "domain name pointer" | cut -d" " -f5
done > reverse_domains.txt
```

3. Practical Step-by-Step Workflow

Scenario: Investigate abcd.com's server neighbors

Step 1: Get the IP address

```bash
# Method A: Using dig
IP=$(dig abcd.com +short | head -1)
echo "IP Address: $IP"
```

```
# Method B: Using ping (shows IP in output)
ping -c 1 abcd.com | grep "PING" | cut -d" " -f3 | tr -d '()'
```

Step 2: Check if it's shared hosting

```
bash
# Check if it's Cloudflare or common CDN (these won't show reverse)
whois $IP | grep -i "cloudflare\|akamai\|fastly\|cloudfront"
```

Step 3: Perform reverse lookup

```
bash
# Method 1: Basic reverse DNS
echo "=== Basic Reverse DNS ==="
host $IP

# Method 2: Using online API (no key required for few requests)
echo "=== Using HackerTarget API ==="
curl -s "https://api.hackertarget.com/reverseiplookup/?q=$IP"

# Method 3: Using Bing/Google dorks via CLI
echo "=== Searching for other sites ==="
curl -s "https://www.bing.com/search?q=ip%3A$IP" | grep -o "http[^\"<>]*" |
sort -u

# Method 4: Check SSL certificates (common on shared hosts)
echo "=== Checking SSL Certificates ==="
curl -s "https://crt.sh/?q=$IP&output=json" | jq -r '.[].name_value' | sort -u
```

Step 4: Advanced enumeration

```
bash
#!/bin/bash
# reverse_ip_investigation.sh

TARGET="abcd.com"
echo "[*] Investigating: $TARGET"
```

```
# Get IP
IP=$(dig $TARGET +short | head -1)
echo "[+] IP Address: $IP"

# Create output directory
mkdir -p reverse_investigation

# 1. Basic reverse DNS
echo "[1] Basic reverse DNS lookup..."
host $IP > reverse_investigation/1_basic_reverse.txt
echo "    Saved to: reverse_investigation/1_basic_reverse.txt"

# 2. Online tools via API
echo "[2] Querying online databases..."
# HackerTarget
curl -s "https://api.hackertarget.com/reverseiplookup/?q=$IP" >
reverse_investigation/2_hackertarget.txt
# ViewDNS (simplified)
curl -s "https://viewdns.info/reverseip/?host=$IP&t=1" | grep -A 100 "Domain"
| grep -oP 'href="[^"]*"' | cut -d'"' -f2 | grep -v viewdns >
reverse_investigation/3_viewdns.txt

# 3. SSL Certificate search
echo "[3] Searching SSL certificates..."
# Install jq if needed: sudo apt install jq
curl -s "https://crt.sh/?q=%25.$IP&output=json" 2>/dev/null | jq -r
'.[].name_value' 2>/dev/null | sort -u > reverse_investigation/4_ssl_certs.txt

# 4. Port scan the IP (light)
echo "[4] Quick port scan..."
nmap -sV --top-ports 20 $IP -oN reverse_investigation/5_nmap_scan.txt >
/dev/null 2>&1

# 5. Check for subdomains of found domains
echo "[5] Enumerating subdomains of neighbors..."
cat reverse_investigation/2_hackertarget.txt
reverse_investigation/3_viewdns.txt reverse_investigation/4_ssl_certs.txt
2>/dev/null | sort -u | grep -v "^$" > reverse_investigation/all_domains.txt

echo ""
echo "=== SUMMARY ==="
echo "IP: $IP"
```

```
echo "Domains on same IP: $(wc -l < reverse_investigation/all_domains.txt)"
echo ""
echo "All found domains:"
cat reverse_investigation/all_domains.txt
echo ""

echo "Files saved in: reverse_investigation/"
```

Step 5: Analyze results for attack surface

bash

```
# Check which domains are live
cat reverse_investigation/all_domains.txt | httpx -no-color -silent >
reverse_investigation/live_domains.txt

# Check for common vulnerabilities
nuclei -l reverse_investigation/live_domains.txt -t ~/nuclei-templates/ -o
reverse_investigation/vulnerabilities.txt

# Look for weak configurations
echo "=== Checking for common issues ==="
for domain in $(cat reverse_investigation/live_domains.txt); do
    echo "Checking: $domain"
    # Check for directory listing
    curl -s "$domain" | grep -i "index of" && echo "[!] Directory listing
enabled: $domain"
    # Check for exposed admin panels
    curl -s "$domain/admin" -I | grep "200" && echo "[!] Admin panel found:
$domain/admin"

done
```

4. Automated Script

bash

```
#!/bin/bash
# reverse_ip_hunter.sh

if [ -z "$1" ]; then
    echo "Usage: $0 domain.com"
```

```
    exit 1
fi

TARGET=$1
OUTPUT_DIR="reverse_scan_${TARGET}"
mkdir -p $OUTPUT_DIR

echo "[*] Starting reverse IP investigation for: $TARGET"
echo "[*] Output directory: $OUTPUT_DIR"

# Get IP
IP=$(dig $TARGET +short | head -1)
if [ -z "$IP" ]; then
    echo "[!] Could not resolve IP for $TARGET"
    exit 1
fi
echo "[+] Target IP: $IP"

# Function to get domains from crt.sh
get_ssl_domains() {
    curl -s "https://crt.sh/?q=%25.$IP&output=json" 2>/dev/null | \
    jq -r '.[].name_value' 2>/dev/null | \
    sed 's/\*\.//g' | sort -u
}

# Function to get domains from hackertarget
get_hackertarget_domains() {
    curl -s "https://api.hackertarget.com/reverseiplookup/?q=$IP" 2>/dev/null
}

# Collect data
echo "[1] Gathering data from multiple sources..."
get_ssl_domains > $OUTPUT_DIR/ssl_domains.txt
get_hackertarget_domains > $OUTPUT_DIR/hackertarget_domains.txt
host $IP 2>/dev/null | grep "domain name pointer" | awk '{print $NF}' | sed
's/\.$//' > $OUTPUT_DIR/reverse_dns.txt

# Combine and deduplicate
cat $OUTPUT_DIR/*.txt 2>/dev/null | sort -u | grep -v "^$" >
$OUTPUT_DIR/all_domains.txt
TOTAL_DOMAINS=$(wc -l < $OUTPUT_DIR/all_domains.txt)
```

```
echo "[+] Found $TOTAL_DOMAINS unique domains on IP $IP"

# Check live domains
echo "[2] Checking which domains are live..."
cat $OUTPUT_DIR/all_domains.txt | httpx -no-color -silent -timeout 5 >
$OUTPUT_DIR/live_domains.txt
LIVE_COUNT=$(wc -l < $OUTPUT_DIR/live_domains.txt)
echo "[+] $LIVE_COUNT domains are live"

# Quick vulnerability check
echo "[3] Quick vulnerability scan..."
if [ $LIVE_COUNT -gt 0 ] && [ $LIVE_COUNT -lt 50 ]; then
    nuclei -l $OUTPUT_DIR/live_domains.txt -silent -severity
medium,high,critical -o $OUTPUT_DIR/vulnerabilities.txt
    VULN_COUNT=$(wc -l < $OUTPUT_DIR/vulnerabilities.txt 2>/dev/null || echo
0)
    echo "[+] Found $VULN_COUNT potential vulnerabilities"
fi

# Generate report
echo ""
echo "=== REVERSE IP INVESTIGATION REPORT ==="
echo "Target: $TARGET"
echo "IP Address: $IP"
echo "Total domains on IP: $TOTAL_DOMAINS"
echo "Live domains: $LIVE_COUNT"
echo ""
echo "All domains found:"
echo "------------------"
cat $OUTPUT_DIR/all_domains.txt
echo ""
echo "Live domains:"
echo "-------------"
cat $OUTPUT_DIR/live_domains.txt 2>/dev/null || echo "None"
echo ""

echo "Files saved in: $OUTPUT_DIR/"
```

5. Real-World Attack Scenario Example

Scenario: You find `vulnerable-site.com` with SQL injection. Same IP hosts `client-portal.com`.

```bash
# Discover they're on same server
dig vulnerable-site.com +short  # Returns: 192.0.2.100
dig client-portal.com +short    # Returns: 192.0.2.100

# Exploit SQLi on vulnerable-site.com
sqlmap -u "http://vulnerable-site.com/page?id=1" --dbs

# If you get shell on vulnerable-site.com:
# 1. Enumerate server
ls -la /var/www/html/
# 2. Find other sites
ls /var/www/  # Shows client-portal/, admin-portal/, etc.
# 3. Access their files
cat /var/www/client-portal/config/database.php
```

6. Important Notes & Limitations

1. CDNs: Cloudflare, Akamai, etc. hide real IPs
2. Rate Limits: APIs have usage limits
3. Accuracy: Not all reverse IP data is complete
4. Legal: Only test domains you own or have permission for
5. Privileged Hosting: Some hosts (AWS, GCP) won't show reverse DNS

7. Quick Commands Reference

```bash
# One-liner to find domains on same IP
dig $(dig target.com +short) -x +short

# Using multiple methods in one go
for ip in $(dig target.com +short); do curl -s
"https://api.hackertarget.com/reverseiplookup/?q=$ip"; done
```

```
# Check if target uses Cloudflare
dig target.com +short | while read ip; do whois $ip | grep -i cloudflare &&
echo "Cloudflare detected: $ip"; done
```

This approach helps you understand the complete attack surface when multiple sites share infrastructure. Always ensure you have proper authorization before testing!